05강

프로그래밍 언어론

구문 분석

컴퓨터과학과 김진욱교수

학습목차



1. 어휘 분석
2. 파스트리
3. 모호성

01

어휘 분석

1. 어휘 분석

프로그램 분석

문자: i, n, t, x, 1, 2, ;, =, +, 5, *, f, >, …

↓ 어휘 분석

어휘: int, x12, ;, =, 1, +, 5, if, >, 10, then, …

↓ 구문 분석

구문: <선언문> ::= <자료형> <변수> ';' ;
<대입문> ::= <변수> = <수식> ';' ;

```
int x12;
x12 = 1 + 5 * 2;
if x12>10 then ...(생략)
```

1. 어휘 분석

어휘 분석

- 목적: 프로그램에서 사용되는 단어를 구별해 냄

토큰(token)

- 어휘 분석을 통해 얻는 결과
- 예: 연산자, 구분자, 식별자, 예약어, 리터럴 등

- 토큰과 렉심
  - 렉심(lexeme): 프로그램에 사용되는 단어
  - 토큰: 렉심의 집합

1. 어휘 분석

식별자

- 변수나 함수 등의 이름을 나타내는 토큰
- 전통적인 식별자: 문자와 숫자로 구성, 첫 글자는 문자

```
<identifier> ::= <letter>
               | <identifier> <letter>
               | <identifier> <digit>
               ;
<letter> ::= A | B | C | D | ... | X | Y | Z
           | a | b | c | d | ... | x | y | z
           ;
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
          ;
```

1. 어휘 분석

예약어

- 프로그래밍 언어 자체에 정의되어 포함된 토큰
- 예: `if`, `for`, `int` 등
- 식별자와 구문 구조가 같지만 식별자로 사용 못함
  - 사용자 재정의 불가
- C 언어에서 어휘 분석기는 예약어를 먼저 검사한 후 예약어를 토큰으로 반환

02

파스트리

2. 파스 트리

유도(derivation)

- 구문 규칙을 이용하여 주어진 프로그램을 만들어 내는 과정
- 유도가 가능하면 문법적 오류가 없는 유효한 프로그램임

예: 수식 1+5*2

```
<exp> ::= <exp> + <exp> | <exp> - <exp>
        | <exp> * <exp> | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
          ;
```



1+5*2

2. 파스 트리

유도(derivation)

예: 수식 1+5*2 유도

```
<exp> ::= <exp> + <exp>
        | <exp> - <exp>
        | <exp> * <exp>
        | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
<digit> ::= 0 | 1 | 2 | 3 | 4
          | 5 | 6 | 7 | 8 | 9
          ;
```

```
<exp>
⇒ <exp>+<exp>
⇒ <exp>+<exp>*<exp>
⇒ <exp>+<exp>*<digit>
⇒ <exp>+<exp>*2
⇒ <exp>+<digit>*2
⇒ <exp>+5*2
⇒ <digit>+5*2
⇒ 1+5*2
```

2. 파스 트리

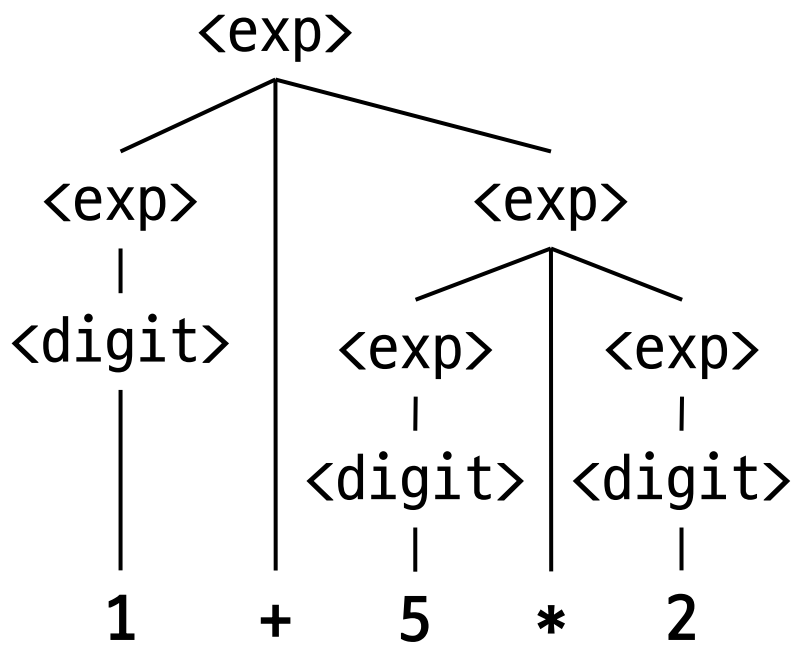
- 파스 트리(parse tree)
  - 유도를 트리 형태로 나타낸 것
  - 구조
    - 루트 노드: 시작 비단말 기호
    - 단말 노드: 단말 기호
  - 단말 노드를 왼쪽부터 오른쪽으로 차례로 나열하면 주어진 프로그램이 됨

2. 파스 트리

파스 트리(parse tree)

예: 수식 1+5*2

```
<exp> ::= <exp> + <exp>
        | <exp> - <exp>
        | <exp> * <exp>
        | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
<digit> ::= 0 | 1 | 2 | 3 | 4
          | 5 | 6 | 7 | 8 | 9
          ;
```



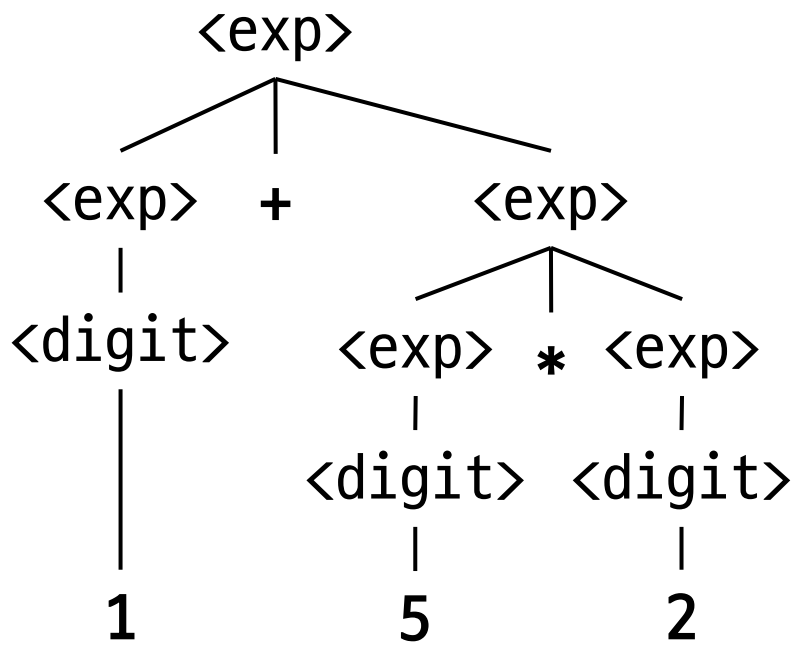
2. 파스 트리

- 파스 트리(parse tree)

예: 수식 1+5*2

```
<exp> ::= <exp> + <exp> | <exp> - <exp>
        | <exp> * <exp> | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
          ;
```

```
<exp>
⇒ <exp>+<exp>
⇒ <exp>+<exp>*<exp>
⇒ <exp>+<exp>*<digit>
⇒ <exp>+<exp>*2
⇒ <exp>+<digit>*2
⇒ <exp>+5*2
⇒ <digit>+5*2
⇒ 1+5*2
```



2. 파스 트리

파스 트리(parse tree)

- 주어진 표현에 대한 파스 트리가 존재

  ➔ 구문에 부합하는 표현

- 주어진 표현에 대한 파스 트리가 존재하지 않음

  ➔ 오류 있는 표현

예: 수식 1+5* ?

```
<exp> ::= <exp> + <exp> | <exp> - <exp>
        | <exp> * <exp> | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
          ;
```

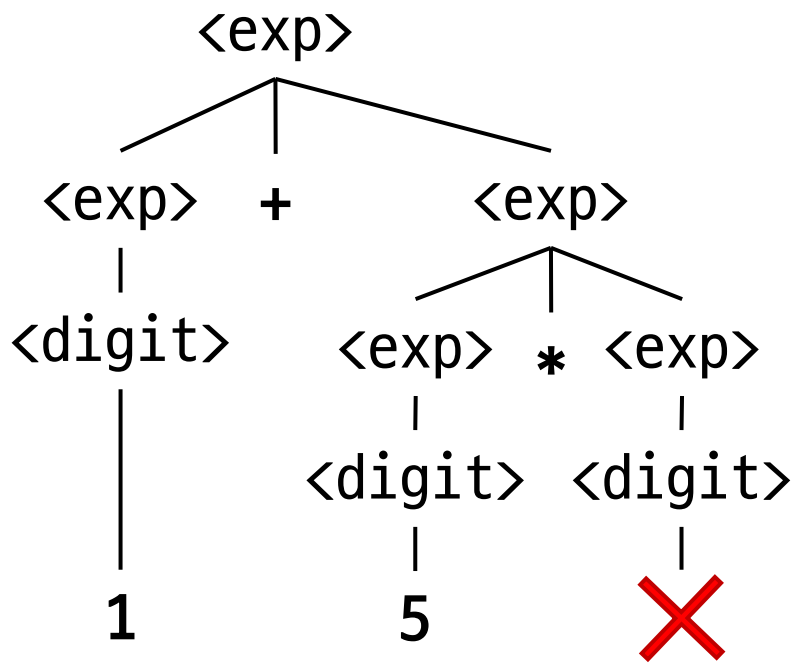
2. 파스 트리

- 파스 트리(parse tree)

예: 수식 1+5*

```
<exp> ::= <exp> + <exp> | <exp> - <exp>
        | <exp> * <exp> | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
          ;
```

<exp>
⇒ <exp>+<exp>
⇒ <digit>+<exp>
⇒ 1+<exp>
⇒ 1+<exp>*<exp>
⇒ 1+<digit>*<exp>
⇒ 1+5*<exp>
⇒ 1+5*<digit>
⇒ ✗



03

모호성

3. 모호성

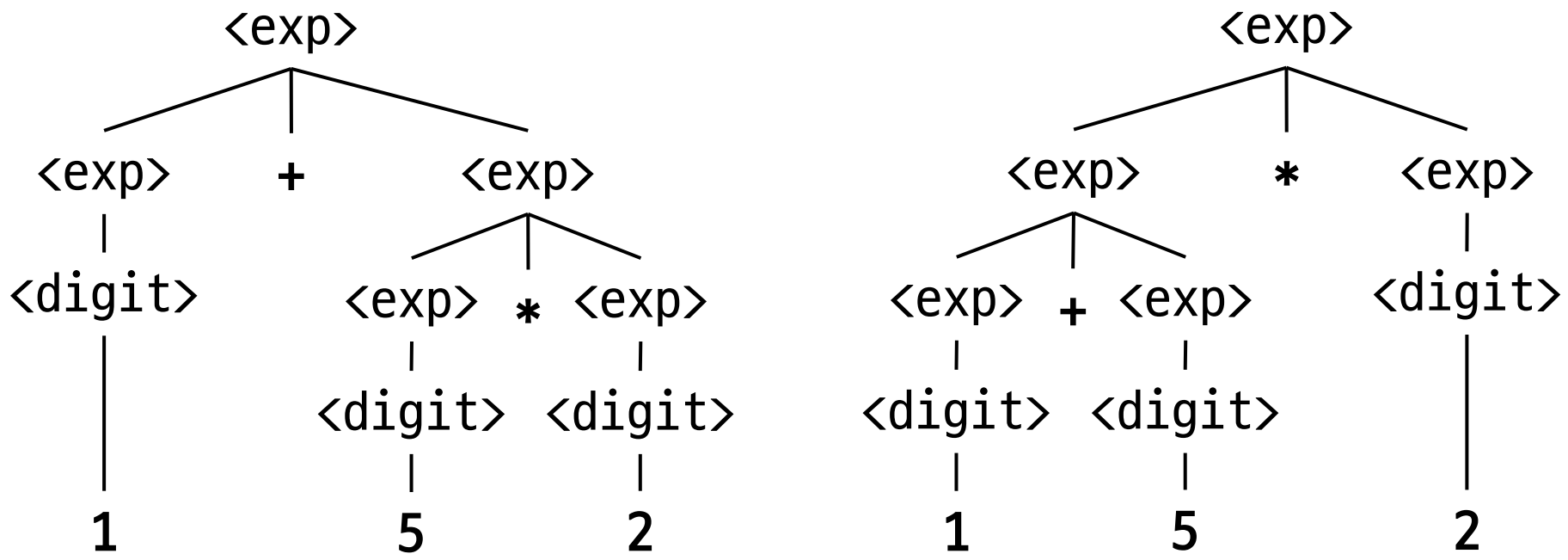
- 파스 트리와 모호성
  - 주어진 표현에 대한 파스 트리가 존재
    - ➔ 구문에 부합하는 표현
  - 만약 주어진 표현에 대한 파스 트리가 여러 개 존재한다면?
    - 구문론 관점
      - ➔ 파스 트리가 존재하므로 구문에는 부합
    - 의미론 관점
      - ➔ 주어진 표현이 서로 다른 의미로 해석될 수 있음

3. 모호성

- 서로 다른 파스 트리

예: 수식 1+5*2

```
<exp> ::= <exp> + <exp> | <exp> - <exp>
        | <exp> * <exp> | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
          ;
```

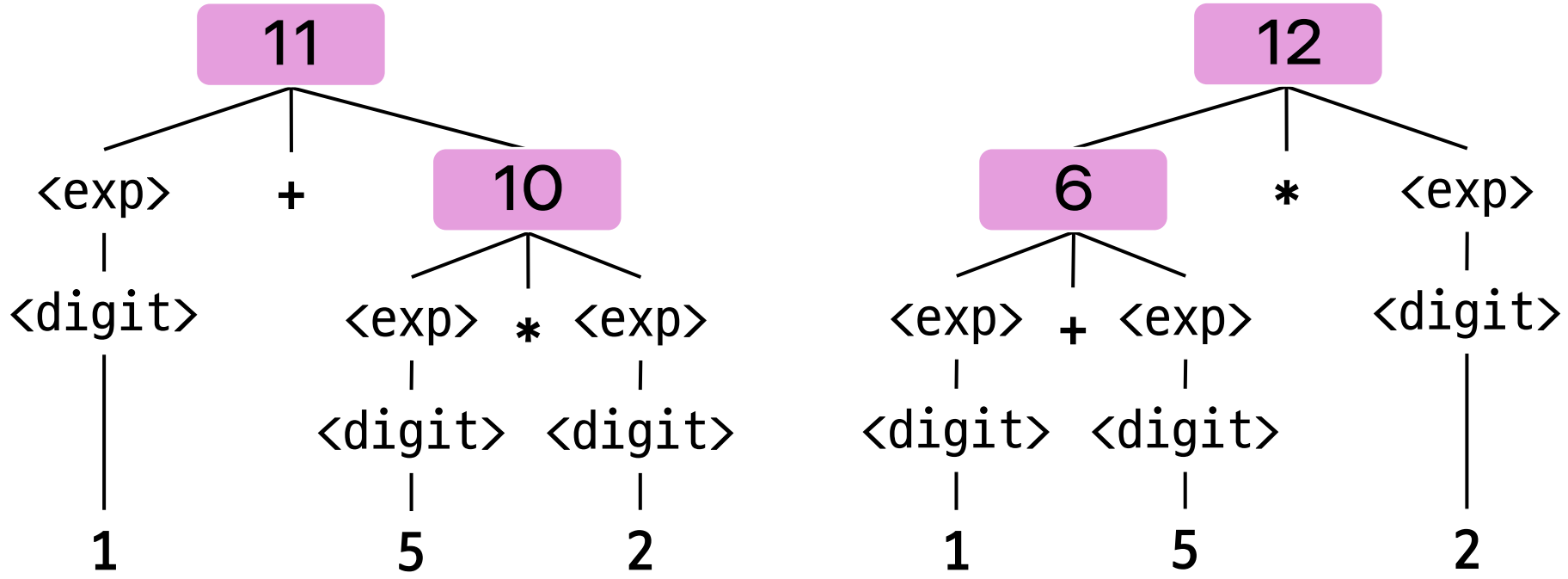


3. 모호성

- 서로 다른 파스 트리

예: 수식 1+5*2

```
<exp> ::= <exp> + <exp> | <exp> - <exp>
        | <exp> * <exp> | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
          ;
```



3. 모호성

모호한 문법

- 동일한 표현에 대해 서로 다른 파스 트리가 만들어지는 문법

문제점

- 하나의 프로그램이 서로 다른 결과를 도출할 수 있음
- 프로그래머의 의도와는 다르게 해석되어 잘못된 결과를 도출할 수 있는 위험을 내포함

3. 모호성

모호성 제거

문법의 명확화

- 의도하지 않은 의미로 해석되지 않도록 모호한 문법을 명확하게 변경
- 새로운 비단말 기호와 새로운 구문 규칙을 추가

대표적인 예

- 연산자 우선순위
- 좌결합 연산자
- 중첩된 `if`문의 `else`

3. 모호성

모호성 제거

연산자 우선순위

- +, – : 가장 낮음
- *, / : 중간
- () : 가장 높음

```
<exp> ::= <exp> + <exp>
        | <exp> - <exp> | <term>
        ;
<term> ::= <term> * <term>
         | <exp> / <exp> | <factor>
         ;
<factor> ::= ( <exp> ) | <digit>
           ;
```

우선순위 계층화

- 모호한 문법

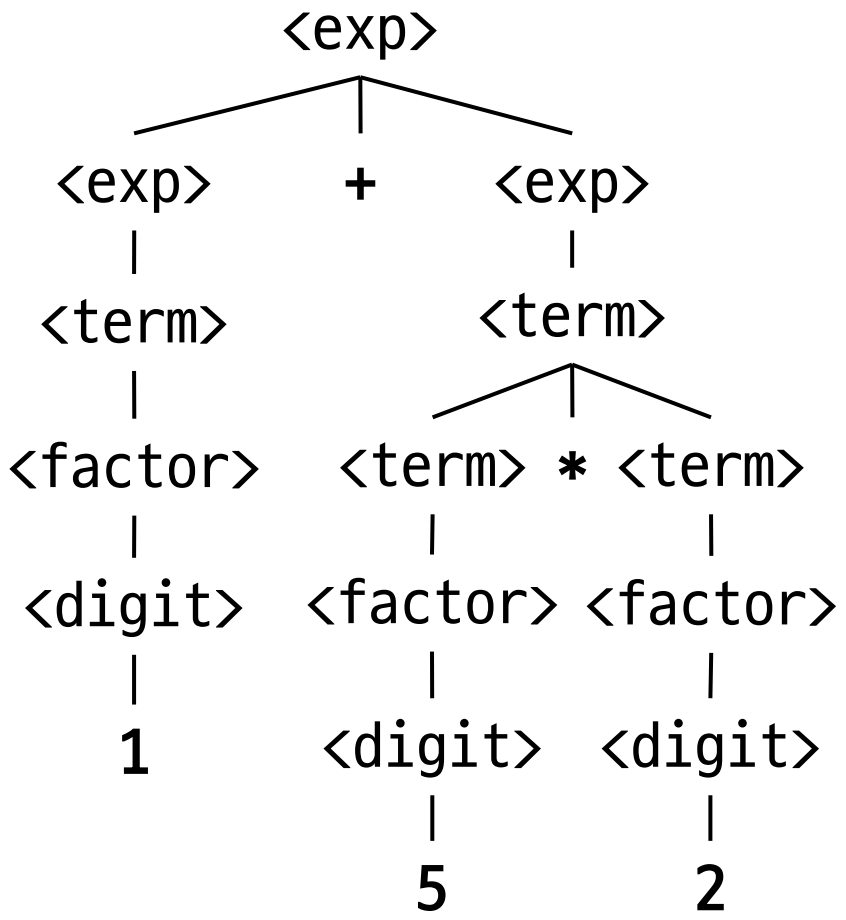
```
<exp> ::= <exp> + <exp>
        | <exp> - <exp>
        | <exp> * <exp>
        | <exp> / <exp>
        | ( <exp> ) | <digit>
        ;
```

3. 모호성

연산자 우선순위

예: 수식 1+5*2

```
<exp> ::= <exp> + <exp>
        | <exp> - <exp>
        | <term>
        ;
<term> ::= <term> * <term>
         | <term> / <term>
         | <factor>
         ;
<factor> ::= ( <exp> )
           | <digit>
           ;
```

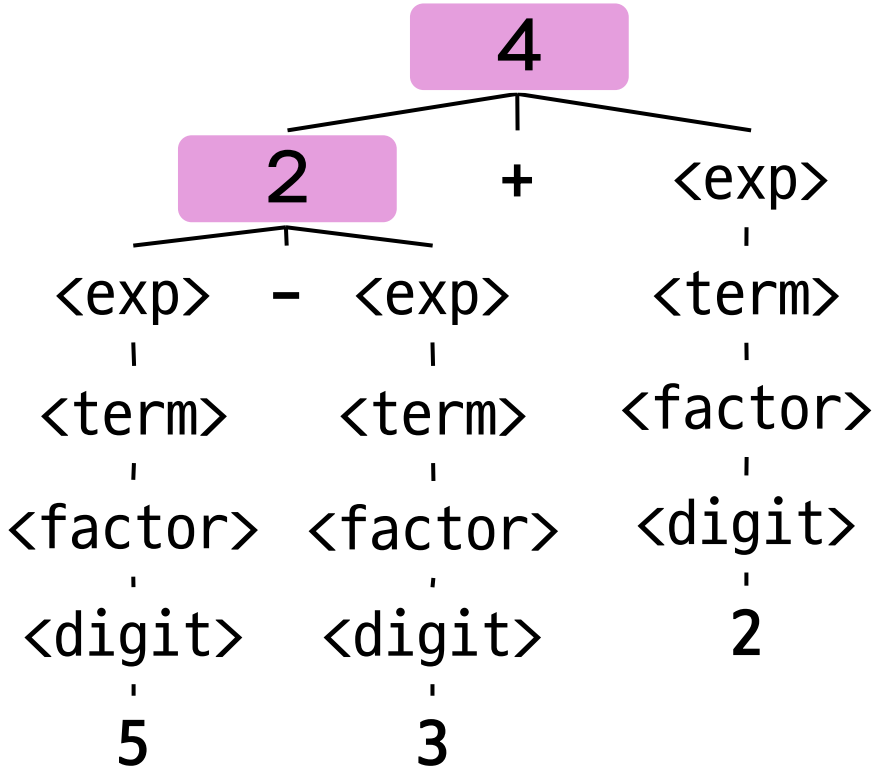


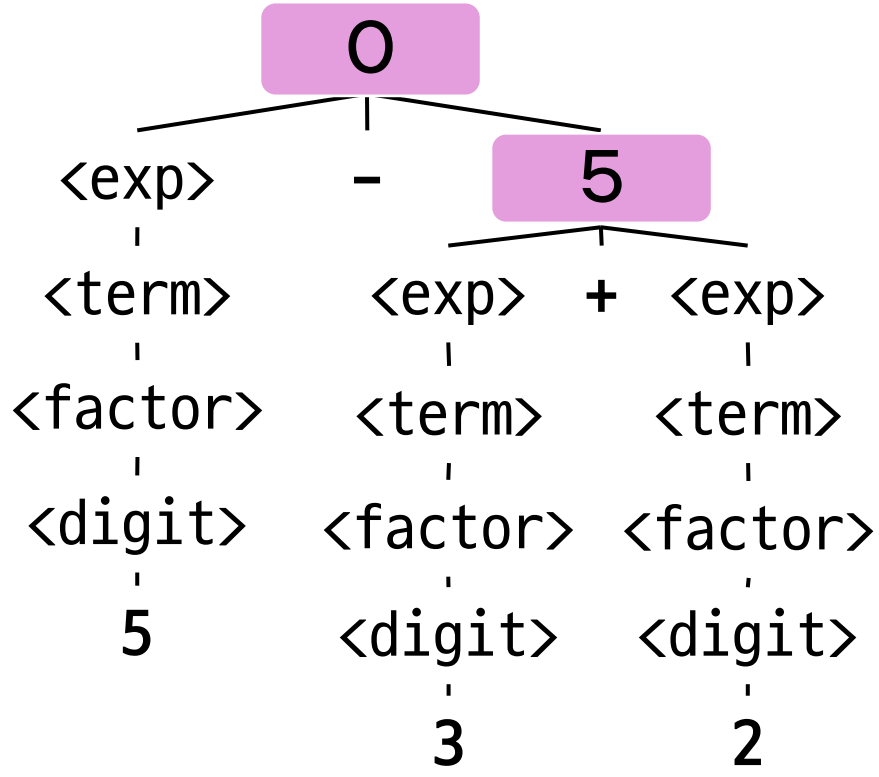
3. 모호성

- 서로 다른 파스 트리

예: 수식 5-3+2

```
<exp> ::= <exp> + <exp> | <exp> - <exp> | <term>
        ;
<term> ::= <term> * <term> | <exp> / <exp> | <factor>
         ;
<factor> ::= ( <exp> ) | <digit>
           ;
```





3. 모호성

모호성 제거

좌결합 연산자

- 우선순위가 동일한 연산자가 피연산자를 사이에 두고 연속된 경우, 왼쪽 연산자를 우선 고려함

- 여전히 모호한 문법

```
<exp> ::= <exp> + <exp>
        | <exp> - <exp> | <term>
        ;
<term> ::= <term> * <term>
         | <exp> / <exp> | <factor>
         ;
<factor> ::= ( <exp> ) | <digit>
           ;
```

```
<exp> ::= <exp> + <term>
        | <exp> - <term> | <term>
        ;
<term> ::= <term> * <factor>
         | <exp> / <factor> | <factor>
         ;
<factor> ::= ( <exp> ) | <digit>
           ;
```

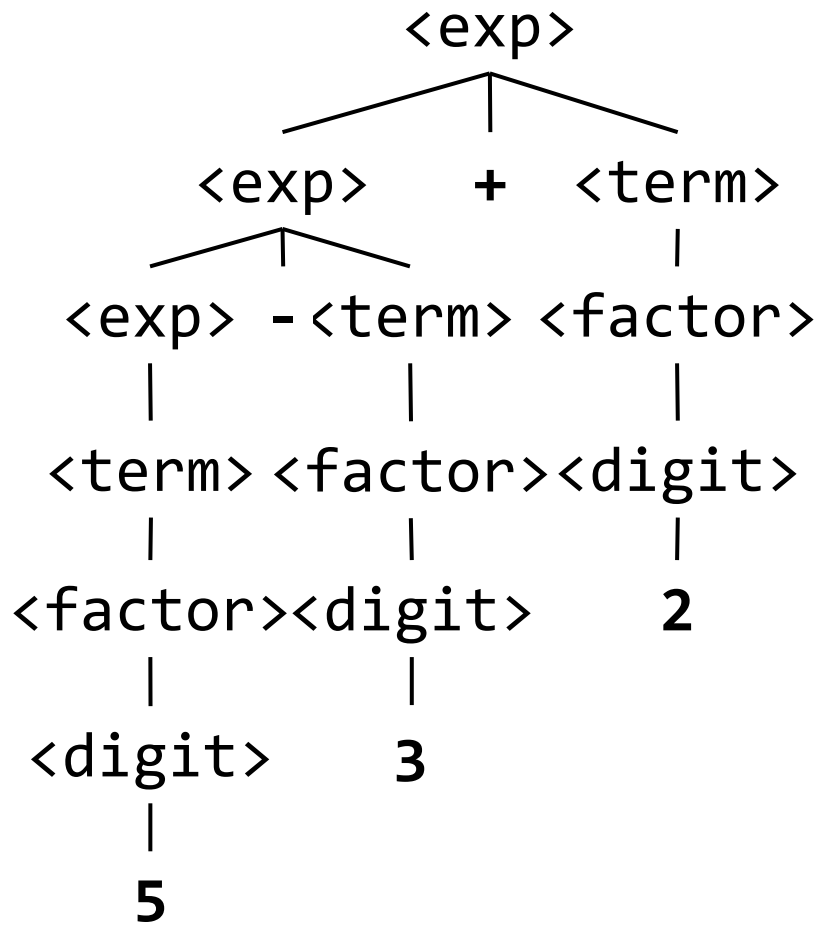
결합 방향 강제

3. 모호성

좌결합 연산자

예: 수식 5-3+2

```
<exp> ::= <exp> + <term>
        | <exp> - <term>
        | <term>
        ;
<term> ::= <term> * <factor>
         | <exp> / <factor>
         | <factor>
         ;
<factor> ::= ( <exp> )
           | <digit>
           ;
```



3. 모호성

짝 잃은 else 문제(dangling else problem)

중첩된 if문의 else

- 중첩된 if문에서 else문의 개수가 if문의 개수보다 적은 경우 각 else문을 어느 조건이 거짓을 때 수행해야 하는지 모호함

- 모호한 문법

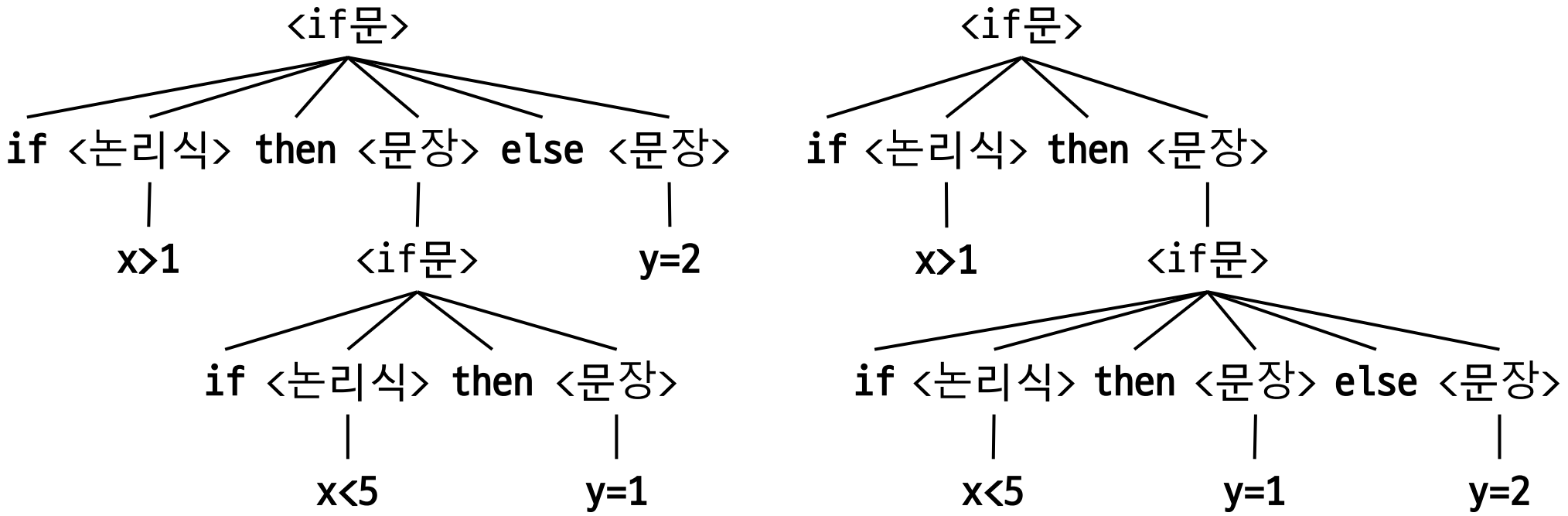
```
<if문> ::= if <논리식> then <문장> else <문장>
         | if <논리식> then <문장>
         ;
<문장> ::= <if문> | <대입문> | ...
         ;
```

3. 모호성

```
<if문> ::= if <논리식> then <문장> else <문장>
         | if <논리식> then <문장>
         ;
<문장> ::= <if문> | <대입문> | ...
         ;
```

서로 다른 파스 트리

예: if x>1 then if x<5 then y=1 else y=2



3. 모호성

❑ 모호성 제거

중첩된 if문의 else

- else문 앞에 나온 if문들 중 다른 else문과 짝이 되지 않은 가장 가까운 if문과 짝이 되도록 정함

- 모호한 문법

```
<if문> ::= if <논
         | if <논
         ;
<문장> ::= <if문
         ;
```

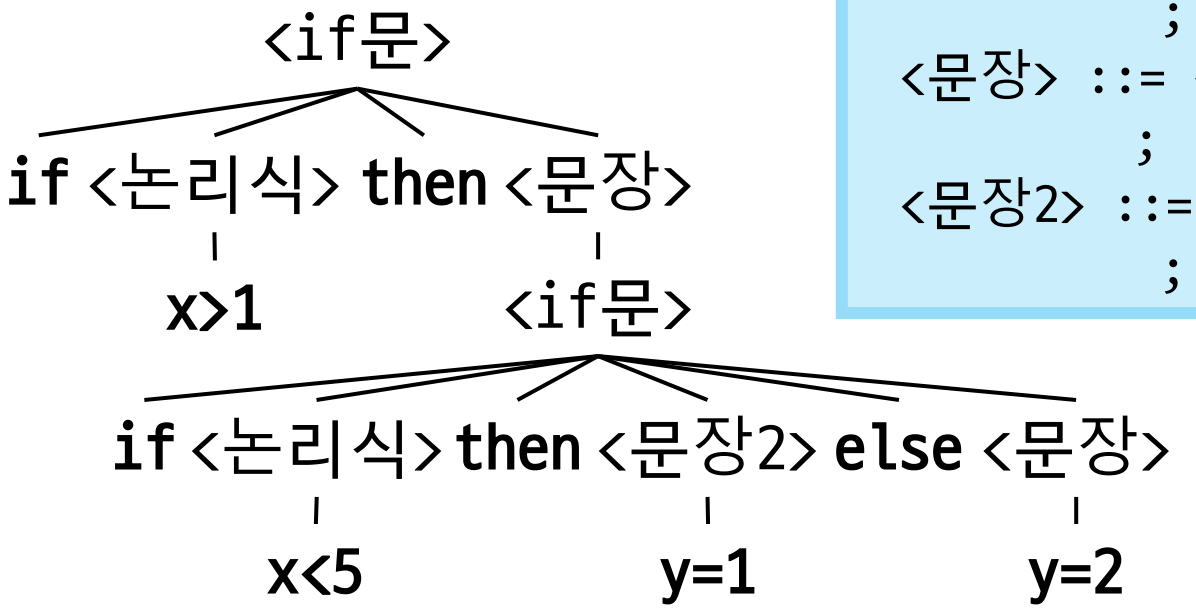
```
<if문> ::= if <논리식> then <문장2> else <문장>
         | if <논리식> then <문장>
         ;
<if문2> ::= if <논리식> then <문장2> else <문장2>
          ;
<문장> ::= <if문> | <문장2> | ...
         ;
<문장2> ::= <if문2> | <대입문> | ...
          ;
```

3. 모호성

중첩된 if문의 else

예: if x>1 then if x<5 then y=1 else y=2

```
<if문> ::= if <논리식> then <문장2> else <문장>
         | if <논리식> then <문장>
         ;
<if문2> ::= if <논리식> then <문장2> else <문장2>
          ;
<문장> ::= <if문> | <문장2> | ...
         ;
<문장2> ::= <if문2> | <대입문> | ...
          ;
```



정리하기

- 프로그램의 분석은 크게 어휘 분석과 구문 분석으로 나눌 수 있음
- 어휘 분석을 통해 프로그램에서 사용된 단어인 토큰을 구별해 냄. 토큰의 종류에는 연산자, 구분자, 식별자, 예약어 등이 있음
- 파스 트리는 유도를 트리 형태로 나타낸 것으로, 루트 노드는 시작 비단말 기호, 단말 노드는 단말 기호가 되며, 단말 노드를 왼쪽부터 오른쪽으로 차례로 나열하면 주어진 프로그램이 됨
- 모호한 문법은 동일한 표현에 대해 서로 다른 파스 트리가 만들어지는 문법으로, 하나의 프로그램이 서로 다른 결과를 도출할 수 있는 위험을 내포함
- 모호한 문법은 문법을 명확하게 변경하여 모호성을 제거해야 하는데, 일반적인 모호성 제거 방법으로는 우선순위 계층화, 결합 방향 강제, 짝 잃은 else 문제 처리법 등이 있음

06강

다음시간안내 ▶▶▶

프로그래밍 언어의 구현